

Análise concorrente do consumo:

Rota Vital / Código Vermelho

1. Justificativa

Operação escolhida

Painel descritivo US06: estatísticas de um histórico de consumo (soma, média, variância, desvio padrão, coeficiente de variação, **mediana e percentil 95**). Na escala nacional, é um relatório que pode atravessar milhões de registros de hospitais.

Candidatas avaliadas

Operação	Onde o tempo é gasto	Decisão
Estatísticas do histórico (mediana e P95 incluídas)	Cálculo e ordenação sobre dados já em memória (CPU)	Escolhida
Cruzamento requisições × estoque	Depende do JOIN e dos índices no banco; a melhoria natural é na consulta SQL	Descartada
Ordenação de filas por prioridade	CPU, $O(n \log n)$, particionável	Válida, mas usa a mesma técnica (ordenar fatias e mesclar)
Deteção de duplicatas	Depende de uma estrutura de hash global, mais sensível a memória	Descartada
Validações em lote	Trabalho por registro muito leve; o custo de coordenar threads tende a superar o ganho	Descartada

Big-O e gargalo

- **Sequencial:** soma e soma dos quadrados são calculadas em uma passada, $O(n)$. Mediana e P95 exigem ordenar o vetor, $O(n \log n)$. O custo dominante é **$O(n \log n)$** .
- **Gargalo:** processamento. Os dados já estão na camada de aplicação e o tempo medido é gasto calculando e ordenando, sem esperar banco ou rede.
- **Particionável:** soma e soma dos quadrados são aditivas entre fatias, e a ordenação pode ser feita por fatias e depois mesclada. Cada tarefa trabalha só na sua fatia contígua, sem escrita compartilhada, então não há race condition. As somas usam *long*, portanto o resultado é exato em qualquer ordem de agregação.
- **Com p threads:** cada fatia é ordenada em $O((n/p) \log(n/p))$, em paralelo, e o merge final custa $O(n \log p)$, em uma única thread.

2. O serviço

Endpoint: **GET /api/v1/analises/consumo**

Parâmetro	Valores	Padrão
registros	de 1 a 5.000.000	100000
modo	sequencial ou threads	sequencial
threads	2, 4 ou 8 (número de fatias)	2

GET <http://localhost:8080/api/v1/analises/consumo?registros=100000&modo=sequencial>

GET <http://localhost:8080/api/v1/analises/consumo?registros=1000000&modo=threads&threads=8>

A resposta traz modo, threads, registros, somaConsumo, media, variancia, desvioPadrao, coeficienteVariacao, mediana, percentil95 e tempoProcessamentoNanos. Mediana e P95 usam interpolação linear sobre o vetor ordenado. Os dados são sintéticos e determinísticos, então todas as versões processam a mesma entrada. O pool de 8 workers é criado uma vez, como bean do Spring, e não entra no tempo medido.

Corretude

O teste `AnaliseConsumoServiceTest` compara todos os campos da resposta entre o modo sequencial e o modo com threads, e passou. Durante o desenvolvimento, esse teste detectou uma divergência no P95 causada por um erro no merge, que foi corrigido. Nas medições, o percentil95 foi idêntico em todas as configurações do mesmo tamanho.

3. Medições

Protocolo: 3 chamadas de aquecimento e 10 chamadas medidas por configuração; usamos a mediana. Máquina com 10 núcleos e 12 processadores lógicos, servidor local, medição por script PowerShell. $\text{Speedup} = \text{tempo sequencial} / \text{tempo com threads}$.

Tempo de processamento no servidor (tempoProcessamentoNanos)

Registros	Seq. (ms)	2 thr. (ms)	Speedup 2	4 thr. (ms)	Speedup 4	8 thr. (ms)	Speedup 8
100.000	2,344	2,090	1,12	1,797	1,30	3,777	0,62
1.000.000	23,431	17,867	1,31	17,481	1,34	20,707	1,13

Tempo de resposta do endpoint (visto pelo cliente)

Registros	Seq. (ms)	2 thr. (ms)	Speedup 2	4 thr. (ms)	Speedup 4	8 thr. (ms)	Speedup 8
100.000	5,875	4,844	1,21	3,926	1,50	7,427	0,79
1.000.000	29,899	24,674	1,21	24,626	1,21	27,425	1,09

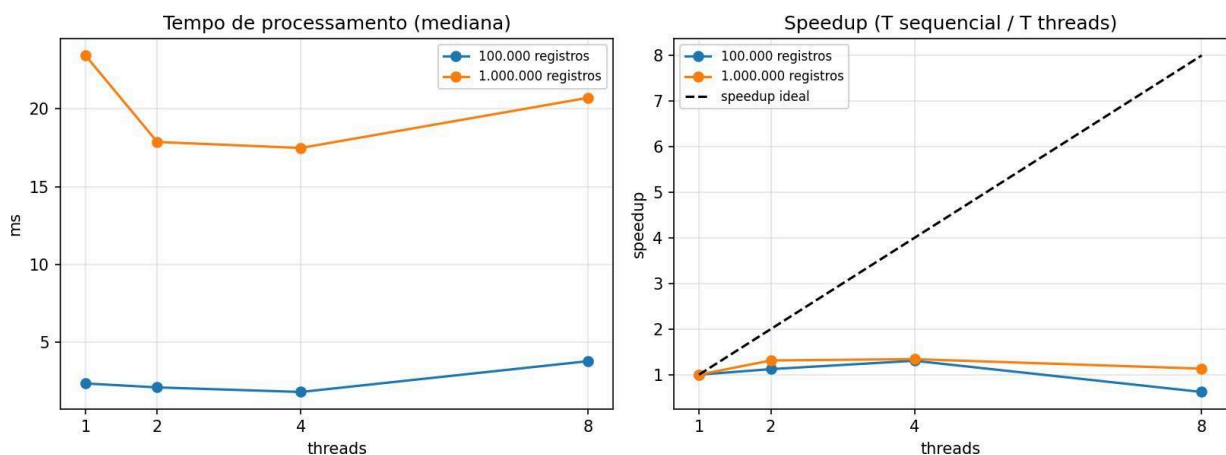


Figura 1. Tempo de processamento (esquerda) e speedup (direita). A linha tracejada é o speedup ideal.

4. Análise

O ganho não foi linear. Com 1 milhão de registros, o melhor caso foi 1,34× com 4 threads (eficiência de cerca de 34%; o ideal seria 4×) e, com 8 threads, caiu para 1,13×. Com 100 mil registros, 8 threads ficou mais lenta que a sequencial (0,62×); como o processamento leva cerca de 2 ms, essas diferenças ficam perto do ruído da medição. A principal explicação é que só parte do trabalho é paralela: ordenar as fatias

roda em paralelo, mas o merge final continua em uma única thread (Lei de Amdahl) e cresce com o número de fatias. Somam-se o custo de despachar tarefas e a disputa por memória, já que ordenar lê e escreve muito. Como a máquina tem 10 núcleos e 12 processadores lógicos, a queda de 4 para 8 threads não se explica por falta de núcleos; nossas hipóteses são o merge maior e a contenção de memória, mas não as isolamos. O tempo total do endpoint mostra o mesmo padrão, com ganho menor, porque geração dos dados, rede local e serialização são custo fixo.

A Big-O não mudou: o trabalho total continua $O(n \log n)$. Cada thread faz $O((n/p) \log(n/p))$ e o merge custa $O(n \log p)$; as threads reduzem o tempo decorrido, não a classe assintótica.

Na Mesa DJ, as threads davam concorrência: vários eventos em andamento, alternando enquanto esperam, o que melhora a responsividade e funciona até com um só núcleo. Aqui é paralelismo: fatias da mesma operação rodam ao mesmo tempo em núcleos diferentes para terminar antes, e o ganho depende dos núcleos e da fração paralelizável. As virtual threads não foram medidas; como servem para esperas bloqueantes e este cálculo usa CPU o tempo todo, não esperamos vantagem sobre as threads de plataforma, mas isso não foi verificado.

Quando 8 threads não bastarem, o caminho é sair de uma única JVM: tirar o relatório da requisição síncrona e usar uma fila de jobs com workers escaláveis horizontalmente, particionados por hospital e período. Soma, contagem e soma dos quadrados combinam-se com facilidade; mediana e P95 exatos exigem mesclar dados ordenados ou usar estimadores aproximados. Completam a evolução uma tabela de agregados atualizada em lote, cache, banco de leitura e observabilidade. Esse é o gancho para a Unidade 2.